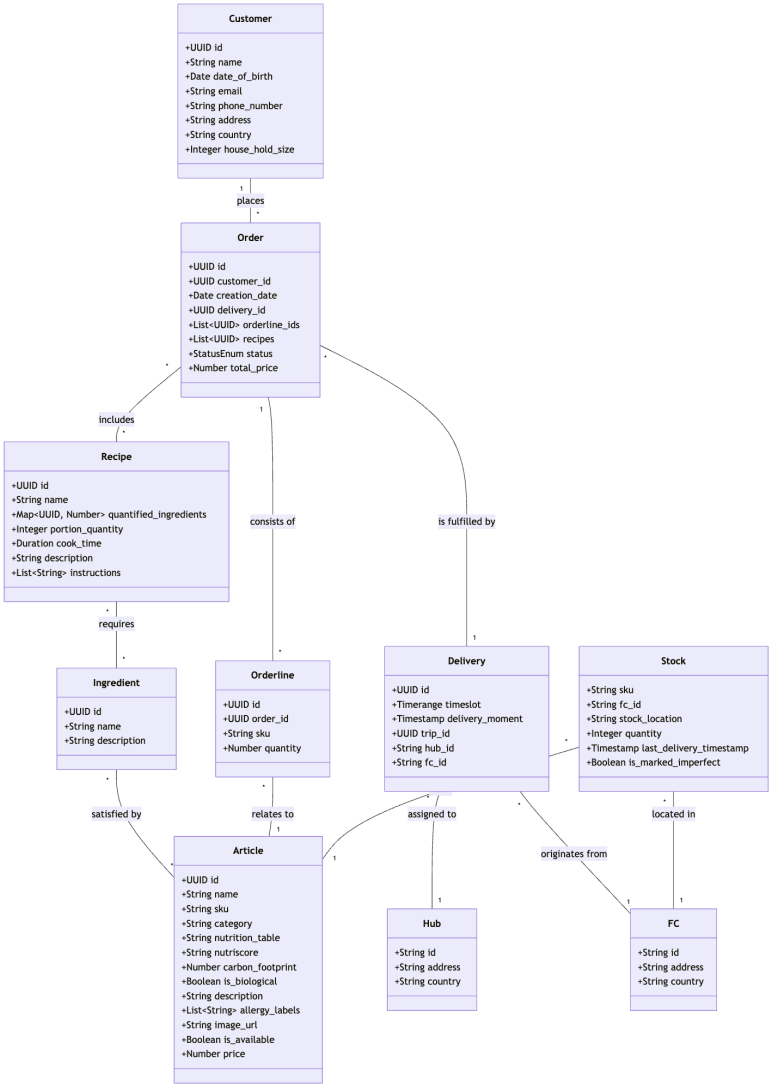
Data models Q-Hackathon 2026

These models are just to help you get started. Feel free to modify or add any new models or fields as you please. The models are purposefully basic to avoid bias and encourage creative solutions.

Placeholder data is used for the examples.

Diagram



Mermaid code for diagram

```
classDiagram
    class Customer {
        +UUID id
```

```
        +String name
        +Date date_of_birth
        +String email
        +String phone_number
        +String address
        +String country
        +Integer house_hold_size
    }

    class Recipe {
        +UUID id
        +String name
        +Map~UUID, Number~ quantified_ingredients
        +Integer portion_quantity
        +Duration cook_time
        +String description
        +List~String~ instructions
    }

    class Ingredient {
            +UUID id
            +String name
            +String description
    }

    class Article {
        +UUID id
        +String name
        +String sku
        +String category
        +String nutrition_table
        +String nutriscore
        +Number carbon_footprint
        +Boolean is_biological
        +String description
        +List~String~ allergy_labels
        +String image_url
        +Boolean is_available
        +Number price
    }

    %% Status: paid → fulfilled → in transit → delivered
    class Order {
            +UUID id
            +UUID customer_id
            +Date creation_date
            +UUID delivery_id
            +List~UUID~ orderline_ids
        +List~UUID~ recipes
        +StatusEnum status
        +Number total_price
```

```
    }

    class Delivery {
        +UUID id
        +Timerange timeslot
        +Timestamp delivery_moment
        +UUID trip_id
        +String hub_id
        +String fc_id
    }

    class Orderline {
        +UUID id
        +UUID order_id
        +String sku
        +Number quantity
    }

    class Stock {
        +String sku
        +String fc_id
        +String stock_location
        +Integer quantity
        +Timestamp last_delivery_timestamp
        +Boolean is_marked_imperfect
    }

    class FC {
        +String id
        +String address
        +String country
    }

    class Hub {
        +String id
        +String address
        +String country
    }

    Customer "1" -- "*" Order : places
    Order "*" -- "*" Recipe : includes
    Recipe "*" -- "*" Ingredient : requires
    Ingredient "*" -- "*" Article : satisfied by
    Order "*" -- "1" Delivery : is fulfilled by
    Order "1" -- "*" Orderline : consists of
    Delivery "*" -- "1" Hub : assigned to
    Delivery "*" -- "1" FC : originates from
    Orderline "*" -- "1" Article : relates to
    Stock "*" -- "1" Article : holds
    Stock "*" -- "1" FC : located in
```

Models

Customer

Represents the customers — users of the app.

Structure:

```
{
  "id": UUID,
  "name": String,
  "date_of_birth": Date,
  "email": String,
  "phone_number": String,
  "address": String,
  "country": String,
  "house_hold_size": Integer, // Number of family members
}
```

Example:

```
{
  "id": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
  "name": "Alex Doe",
  "date_of_birth": "1990-05-15",
  "email": "alex.doe@example.com",
  "phone_number": "+49 123 456789",
  "address": "123 Main St, Exampletown",
  "country": "Germany",
  "house_hold_size": 4
}
```

Recipe

Contains information about recipes, including ingredients and instructions.

Structure:

```
{
  "id": UUID,
  "name": String,
  "quantified_ingredients": Map~UUID, Integer~, // Ingredient ID to quantity
  "portion_quantity": Integer, // E.g., serves 2 persons
  "instructions": List~String~, // Step by step instructions
}
```

Example:

```
{
  "id": "b2c3d4e5-f6a7-8901-2345-67890abcdef0",
  "name": "Simple Salad",
  "description": "A fresh and crunchy starter.",
  "cook_time": "PT15M",
  "quantified_ingredients": {
      "f1e2d3c4...": 1, // one crop of lettuce
      "e2d3c4b5...": 2  // two tomatos
  },
  "portion_quantity": 2,
  "instructions": [
    "Wash vegetables.",
    "Chop vegetables.",
    "Mix in a bowl."
  ]
}
```

Ingredient

Represents an abstract ingredient, such

Structure:

```
{
  "id": UUID,
  "name": String,
  "description": String
}
```

Example:

```
{
  "id": "0fc21cc1-33b2-45d1-abe6-3e8d34fc75d7",
  "name": "Skimmed milk",
  "description": "Milk with low fat percentage."
}
```

Article

Represents the individual items sold in the store.

Structure:

```
{
  "id": UUID,
  "name": String,
  "sku": String, // Stock Keeping Unit, links to the physical item in warehouse
  "category": String,
```

```
  "nutrition_table": String,
  "nutriscore": String,
  "carbon_footprint": Number, // Assuming a numerical value
  "is_biological": Boolean,
  "description": String,
  "allergy_labels": List~String~, // Allergy warnings
  "image_url": String, // URL to product image
  "is_available": Boolean // Inferred from relationship to Stock
  "price": Number
}
```

Example:

```
{
  "id": "c3d4e5f6-a7b8-9012-3456-7890abcdef01",
  "name": "Romaine lettuce",
  "sku": "VEG-LET-001",
  "category": "Vegetables",
  "nutrition_table": "Calories: 15, Fat: 0.2g, Carbs: 2.9g, Protein: 1.4g (per 100g)",
  "nutriscore": "A",
  "carbon_footprint": 0.5, // Example value (e.g., kg CO2e)
  "is_biological": true,
  "description": "Crisp iceberg lettuce, perfect for salads and sandwiches.",
  "allergy_labels": [],
  "image_url": "https://placehold.co/300x300/a2d9a1/333333?text=Lettuce",
  "is_available": true,
  "price": 0.75
}
```

Order

Represents an order placed by a customer that will be delivered.

Structure:

```
{
  "id": UUID,
  "customer_id": UUID,
  "creation_date": Date,
  "delivery_id": UUID,
  "orderline_ids": List~UUID~,
  "recipes": List~UUID~, // Optional: Links to any recipes ordered
  "status": StatusEnum, // paid, fulfilled, in transit, delivered
  "total_price": Float
}
```

Example:

```
{
  "id": "cb8c20e2-3a8a-46e4-ae6b-0451021a2cbf",
```

```
  "customer_id": "a1b2c3d4-e5f6-7890-1234-567890abcdef",
  "creation_date": "2026-04-07T09:15:00Z",
  "delivery_id": "d4e5f6a7-b8c9-0123-4567-890abcdef012",
  "orderline_ids": [
    "e5f6a7b8-c9d0-1234-5678-90abcdef0123",
    "f1a2b3c4-d5e6-4789-8901-234567abcdef"
  ],
  "recipes": [
    "b2c3d4e5-f6a7-8901-2345-67890abcdef0"
  ],
  "status": "paid",
  "total_price": 28.50
}
```

Orderline

Represents a single item line within an order. It may concern multiple of the same items. It's comparable to one line on a receipt.

Structure:

```
{
  "id": UUID,
  "order_id": UUID, // Links back to the Order
  "sku": String, // Links to the Article ordered
  "quantity": Integer
}
```

Example:

```
{
  "id": "e5f6a7b8-c9d0-1234-5678-90abcdef0123",
  "order_id": "cb8c20e2-3a8a-46e4-ae6b-0451021a2cbf",
  "sku": "VEG-LET-001", // Lettuce ID
  "quantity": 2
}
```

Delivery

Represents the trip and actual delivery of an order.

Structure:

```
{
  "id": UUID, // Main ID for the delivery
  "timeslot": Timerange, // e.g., 2025-04-23 10:00-12:00
  "delivery_moment": Timestamp, // Actual moment of delivery
  "trip_id": UUID, // Groups orders that are delivered within the same trip
  "hub_id": String, // Local distribution hub
```

```
  "fc_id": String, // Fulfillment Center
}
```

Example:

```
{
  "id": "d4e5f6a7-b8c9-0123-4567-890abcdef012",
  "timeslot": "2026-04-08 10:00-12:00",
  "delivery_moment": "2026-04-08T11:15:20Z",
  "trip_id": "f6a7b8c9-d0e1-2345-6789-0abcdef01234"
  "hub_id": "HUB-VIE-01",
  "fc_id": "FC-FRA-0
}
```

Stock

Represents the stock level of articles at a specific fulfillment center.

Structure:

```
{
  "sku": String, // Links to Article's SKU
  "fc_id": String, // FC where stock is located
  "stock_location": String, // Specific location inside FC
  "quantity": Integer, // Current stock quantity
  "last_delivery_timestamp": Optional~Timestamp~, // Date + time the product needs to be
delivered before to ensure freshness guarantee
  "is_marked_imperfect": Boolean // If the stock is flagged as imperfect, e.g. dented can
s
}
```

Example:

```
{
  "sku": "VEG-LET-001", // Lettuce SKU
  "fc_id": "FC-FRA-01",
  "stock_location": "BUFF-3-CHILLED",
  "quantity": 42,
  "last_delivery_timestamp": "2025-04-28T12:00:00Z", // ISO 8601 Format
  "is_marked_imperfect": false
}
```

FC

A fulfillment center is where orders for customers get picked into bags in totes, to be sent off towards hubs.

Structure:

```
{
  "id": String,
```

```
  "address": String,
  "country": String,
}
```

Example:

```
{
  "id": "FC-FRA-01",
  "address": "Am Terminal 1, Frankfurt",
  "country": "Germany"
}
```

Hubs

Starting at hub locations, electric Picnic vehicles (EPVs) travel to deliver groceries to neighborhoods in short trips.

Structure:

```
{
  "id": String
  "address": String,
  "country": String,
}
```

Example:

```
{
  "id": "HUB-VIE-01",
  "address": "Industriestraße 5, Viernheim",
  "country": "Germany"
}
```